- PUSL3190 Computing Project
- Abstract
- Chapter 01: Introduction
  - 1.1 Problem Definition
  - 1.2 System Objectives
- Chapter 02: System Analysis and User Research
  - [2.1 Fact Gathering and Domain Investigation
    - Literature Review
  - 2.2 Existing Manual and Digital Tracking Systems
  - [2.3 User Research: Financial Behaviors of Emerging
  - [2.4 Competitive Analysis of the Personal Finance
  - 2.5 Identified Gaps and Behavioral Opportunities
- Chapter 03: Requirements Specification
  - 3.1 Functional Requirements
  - 3.2 Non-Functional Requirements
  - 3.3 Software Requirements
  - 3.4 Hardware Requirements
  - 3.5 Accessibility Standards: WCAG 2.1 AA Compliance
- [Chapter 04: Feasibility Study and Strategic
  - 4.1 Operational and Technical Feasibility
  - [4.2 Economic Feasibility: Comprehensive
  - 4.3 Risk Management and Mitigation Strategies
  - 4.4 Schedule Feasibility and Timeline Practicality
- Chapter 05: System Architecture
  - 5.1 Use Case Diagram
    - Use Case Diagram Assumptions
  - [5.2 Class
    - Class Diagram Assumptions
  - [5.3 High-Level
    - Architectural Assumptions
- [Chapter 06: Development Tools and Technology
  - 6.1 Development Methodology: Agile Scrum Framework
  - [6.2 The MERN Stack: An
  - [6.3 Frontend Architecture and
  - 6.4 Backend and Persistence Layers
  - [6.5 Algorithmic Foundations of the
    - 6.5.1 Expense Forecasting: Linear Regression

    - [6.5.2 Anomaly Detection: Statistical Z-Score
    - [6.5.3 Financial Health Score: Multi-Factor Weighted
    - 6.5.4 Loan EMI Calculator: Banking-Standard Formula
- [Chapter 07:
  - 7.1 Development Environment Setup
  - [7.2
  - [7.3
    - 7.3.1 Backend Code Snippets

—

Description automatically generated]
(file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image002.jpg) |

PUSL3190 Computing Project

Project Interim Report

Design and Implementation of the Smart Financial Manager (SFT) Platform

Supervisor: `<Supervisor Name Here>`

Name: Nethusara Mervin

Plymouth Index Number: 10953504

Degree Program: Computer science

Table of Contents

Abstract5

Chapter 01: Introduction. 6

1.1 Problem Definition. 6

1.2 System Objectives. 7

Chapter 02: System Analysis and User Research. 8

2.1 Fact Gathering and Domain Investigation (Polished)8

Literature Review.. 8

2.2 Existing Manual and Digital Tracking Systems. 9

2.3 User Research: Financial Behaviors of Emerging Adults. 9

2.4 Competitive Analysis of the Personal Finance Ecosystem.. 10

2.5 Identified Gaps and Behavioral Opportunities. 11

Chapter 03: Requirements Specification. 12

3.1 Functional Requirements and Detailed User Stories. 12

3.2 Non-Functional Requirements and Technical Benchmarks. 13

3.3 Hardware and Software Requirements. 13

3.4 Accessibility Standards: WCAG 2.1 AA Compliance. 13

Chapter 04: Feasibility Study and Strategic Assessment14

4.1 Operational and Technical Feasibility. 15

4.2 Economic Feasibility: Comprehensive Cost-Benefit Analysis. 15

4.3 Risk Management and Mitigation Strategies. 15

4.4 Schedule Feasibility and Timeline Practicality. 16

Chapter 05: System Architecture. 17

5.1 Use case diagram.. 17

Use Case Diagram Assumptions. 17

5.2 Class Diagram of Proposed System.. 19

Class Diagram Assumptions. 19

5.3 High-Level Architectural Diagram.. 22

Architectural Assumptions. 23

5.4 Networking Diagram.. 24

Chapter 06: Development Tools and Technology Justification. 25

6.1 Development Methodology: Agile Scrum Framework. 25

6.2 The MERN Stack: An Architectural Defense. 27

6.3 Frontend Architecture and Rendering Strategy. 27

6.4 Backend and Persistence Layers. 28

6.5 Algorithmic Foundations of the SFT Platform.. 28

6.5.1 Expense Forecasting: Linear Regression. 28

6.5.2 Anomaly Detection: Statistical Z-Score Analysis. 28

6.5.3 Financial Health Score: Multi-Factor Weighted Algorithm.. 29

6.5.4 Loan EMI Calculator: Banking-Standard Formula. 29

Chapter 07: Implementation Progress. 30

7.1 Development Environment Setup. 30

7.2 Implemented Features. 30

7.3 Screenshots and Code Snippets. 32

7.3.1 Backend Code Snippets. 32

7.3.2 Frontend Code Snippets. 35

7.3.3 Database Schema Examples. 38

Explanation. 38

7.3.4 User Interface Screenshots. 39

7.4 Challenges Encountered and Solutions. 45

7.5 Current System Limitations. 47

Concluding Note. 47

Chapter 08: Discussion. 48

Summary of the Report48

What Has Changed from the Proposal48

Future Plans and Upcoming Work. 48

References . 49

APPENDIX.. 52

A: Interview Research Materials. 52

A.1 Interview Protocol52

A.2 Core Interview Questions. 52

A.3 Sample Participant Excerpt53

A.4 Key Findings. 53

Table 1: Contextual Factors Influencing Digital Spend. 5

Table 2: Problem Area Impact Mapping. 6

Table 3: Student Financial Hardship Statistics (2024-2025)9

Table 4: Competitive Feature Matrix: SFT vs. Industry Leaders. 10

Table 5: Pain Point Analysis of Automated Budgeting Tools. 11

Table 6: Detailed User Stories and Behavioral Goals. 12

Table 7: Quantitative Non-Functional Requirement Metrics. 13

Table 8: WCAG 2.1 AA Accessibility Success Criteria for Dashboards. 14

Table 9: Production Environment Infrastructure Cost Projection. 15

Table 10: Project Risk Assessment and Impact Matrix. 16

Table 11 Project Risk Assessment and Impact Matrix. 16

Table 12: Sprint Progress Tracker25

Table 13: Comparative Analysis of Backend Frameworks (2025)27

Table 14: Frontend Framework Selection Rationale. 27

Figure 1: Use Case Diagram.. 17

Figure 2: Class Diagram of Proposed System.. 19

[Figure 3: High-Level Architectural Diagram.. 22]
(file:///D:\interim%20report.docx#_Toc223398436)

Figure 4: Networking Diagram.. 24

Figure 5: Database collections within the MongoDB Atlas cluster.38

Figure 6: Database collections within the MongoDB Atlas cluster.38

Figure 7Authentication interface supporting credential-based login and persistent session handling.39

Figure 8: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts (1).40

Figure 9: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts. (2)40

Figure 10: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts. (3)41

Figure 11: Transaction management interface with dynamic filtering (date range, category) and paginated result rendering.41

Figure 12 :Category-specific budget tracking with real-time percentage utilization indicators.42

Figure 13: generate budget from income. 42

Figure 14: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button. 43

Figure 15: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button. 43

Figure 16: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button. 43

Figure 17: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button. 44

Figure 18: Dark theme implementation demonstrating visual consistency across UI components.44

Abstract

The contemporary financial landscape is characterized by rapid transition toward digital ecosystems, where cashless transactions and subscription-based services have significantly reshaped consumer behavior. While these developments have improved transactional convenience, they have simultaneously introduced challenges in financial self-regulation and expenditure awareness. Individuals now manage multiple income streams and recurring expenses across diverse digital platforms, resulting in fragmented financial visibility and reduced control over personal cash flow. Global economic volatility, including inflation fluctuations and uncertain interest rate movements, further intensifies financial decision-making pressures, with choices frequently influenced by emotional responses rather than objective income assessment. The increased reliance on online platforms has fundamentally reduced the physical awareness of spending, creating a disconnect between consumption and consciousness of expenditure. This digital transformation has created an environment where traditional financial tracking methods prove inadequate, and the lack of structured tools contributes to overspending among emerging adults and non-technical users who navigate complex financial landscapes without proper guidance. The Smart Financial Manager (SFT) platform is proposed as a comprehensive web-based personal finance management solution to address these structural and behavioral inefficiencies. Developed using the MERN technology stack—MongoDB, Express.js, React, and Node.js—the system aims to deliver a unified and behaviorally informed environment for financial tracking and budgeting. Unlike conventional banking applications that primarily provide retrospective transaction summaries, the SFT platform emphasizes active financial engagement through structured manual recording and interactive data visualization. Active recording of transactions is linked to stronger financial outcomes than passive automation, as behavioral finance research demonstrates that active cognitive involvement in financial tracking enhances accountability and supports sustainable spending discipline. Manual transaction entry increases awareness of expenditure patterns and reduces the likelihood of impulsive financial decisions associated with present bias, which prioritizes short-term gratification over long-term financial stability. Modern technology stacks such as MERN enable real-time feedback loops essential for financial awareness, allowing users to receive immediate insights into their spending behavior and budget utilization. By

combining behavioral engagement principles with contemporary web technologies, the SFT platform seeks to restore cognitive awareness of spending while providing the analytical power necessary for informed financial decision-making, ultimately empowering users to achieve greater financial control and stability in an increasingly digital economy.

Chapter 01: Introduction

The rise of digital payments has fundamentally altered expenditure awareness, creating an "invisible economy" where contactless transactions remove the psychological friction of spending (OECD, 2024).This "payment decoupling" phenomenon masks true lifestyle costs, leading to financial liabilities that only become apparent at critical thresholds. The SFT platform reintroduces cognitive engagement by requiring manual transaction logging and centralizing all financial commitments into a real-time dashboard, thereby recoupling consumption with immediate awareness.

1.1 Problem Definition

Contemporary personal finance management suffers from limited visibility due to structural weaknesses in traditional tracking approaches, including manual spreadsheets that are prone to errors and lack real-time synchronization (Pressman & Maxim, 2020).When individuals cannot visualize their financial position clearly, budgeting becomes reactive rather than proactive, contributing to household stress and short-term instability (World Bank, 2024).

| **Problem Area** | **Specific Limitation** | **Impact

on the User**	
Data Integrity	Spreadsheets are prone to formula
errors and data loss.	Inaccurate assessment of financial
position.	
Predictive Insight	Traditional tools lack forecasting
capabilities.	Inability to anticipate financial

on the User**	
strain.	
User Experience	Commercial applications are
frequently feature-heavy and complex.	User abandonment and return to
inefficient manual tracking.	
Collaboration	Single-user systems lack integrated
P2P functionality.	Difficulty managing shared expenses.
Proactivity	Alerts occur only after budget limits
are exceeded.	Reduced opportunity for mid-cycle
behavioural correction.	

Table 2: Problem Area Impact Mapping

Commercial financial applications often prioritize automation over behavioral engagement, leading to user disengagement and reduced awareness of spending behaviors (Federal Reserve , 2024). Most tools operate reactively without predictive intelligence, and fragment financial functions across separate platforms, creating a disjointed experience that discourages sustained engagement (Alea IT Solutions, 2025). The proliferation of "Buy Now, Pay Later" (BNPL) services and micro-subscriptions has complicated financial visibility by fragmenting small expenses across time. SFT addresses this through a dedicated recurring liabilities category that displays annualized impacts, helping users identify redundant expenses and improve monthly cash flow.

1.2 System Objectives

To design, implement, and validate a secure, scalable, and behaviorally informed web-based personal finance management system using the MERN technology stack that enhances financial visibility and promotes proactive budgeting through active user engagement. The specific objectives are:

· To develop a comprehensive transaction management system that enables users to create, view, update, and delete income and expense records across eighteen distinct expenditure categories in a structured and reliable manner.

· To implement an intelligent budgeting mechanism with progressive threshold alerts at 80%, 90%, and 100% utilization levels that provide timely notifications to encourage behavioral adjustment and prevent overspending before budget limits are exceeded.

· To create a milestone-based savings goal tracking system with interactive visual progress indicators that enhance user motivation and support achievement of personal financial targets.

· To integrate predictive analytics capabilities using statistical linear regression algorithms that analyze historical spending patterns and forecast expenditure trends to support informed financial planning.

· To establish a secure internal wallet system that enables peer-to-peer transfers with transactional atomicity, double-entry accounting principles, and complete audit trails to ensure data integrity and financial accuracy.

· To deploy multi-tier security through role-based access control and JWT authentication mechanisms across Super Admin, Admin, User, and Guest roles, ensuring appropriate access permissions and data protection for all user types.

· To present financial information through personalized dashboards and data visualizations that transform complex financial data into actionable insights, supporting informed decision-making and improving user confidence in financial management.

· To complete system development, testing, deployment, and comprehensive documentation within the defined project schedule, ensuring readiness for final academic evaluation by April 2026.

Chapter 02: System Analysis and User Research

The analysis phase investigated current personal finance practices and evaluated existing solution limitations to establish functional baselines and identify technological behavioral gaps (Pressman & Maxim, 2020).

[2.1 Fact Gathering and Domain Investigation

(Polished)]()

A mixed-methods research approach was adopted, incorporating literature review (2023–2025), stakeholder interviews, and competitive benchmarking. This triangulated methodology ensured alignment with contemporary personal finance management (PFM) research and industry best practices.

Interview-Based Research

Semi-structured interviews were conducted with 26 stakeholders: 15 university students (aged 19–24), 8 early-career professionals (1–3 years post-graduation), and 3 financial educators. Sessions lasting 20–30 minutes were conducted between December 2025 and January 2026 via in-person and video conferencing platforms.

The interview protocol explored five thematic domains:

- Current financial tracking practices
- Pain points with existing systems
- Desired application features
- Behavioural spending patterns
- Technological comfort and privacy concerns

The semi-structured format permitted thematic exploration while maintaining comparability across participants. A complete list of interview questions and detailed research methodology is provided in Appendix A.

Key Findings

- 73% of students preferred manual tracking (notebooks or spreadsheets) due to privacy concerns regarding bank integration.
- 100% of participants reported dissatisfaction with reactive notifications that occur only after overspending.
- 87% utilised separate applications for budgeting, P2P transfers, and savings tracking, spending 15–30 minutes weekly reconciling data.
- 68% underestimated recurring subscription costs, with average miscalculation of 34%.

Visual Representation of Key Findings

Finding 1: Financial Tracking Preferences

```
Parse error on line 2:
...eWidth': '2px'}}}%%pie title Student Fi
----------------------^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

Finding 2: Satisfaction with Notification Systems

```
Parse error on line 2:
...eWidth': '2px'}}}%%pie title User Satis
----------------------^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

Finding 3: Application Usage Patterns

```
Parse error on line 2:
...eWidth': '2px'}}}%%pie title Financial
----------------------^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

Finding 4: Recurring Subscription Cost Awareness

```
Parse error on line 2:
...eWidth': '2px'}}}%%pie title Recurring
----------------------^
Expecting 'NEWLINE', 'SPACE', 'GRAPH', got 'ALPHA'
```

These findings directly informed SFT's manual-entry emphasis, progressive budget alerts, integrated wallet system, and recurring expense aggregation with annualised cost projections.

Literature Review

Primary research findings demonstrate that financial education is most effective when supported by continuous engagement tools (Xiao & O'Neill, 2018), while Thaler and Sunstein argue that behavioral nudging mechanisms can influence financial decisionmaking by altering choice architecture without restricting freedom (Thaler & Sunstein, 2008).Competitive benchmarking against Monarch Money, YNAB, and PocketGuard identified common pain points: excessive advertising, complex bank synchronization, and usability barriers discouraging long-term adoption (NerdWallet,

2026). Domain expert consultation validated best practices in financial accounting and data security, ensuring P2P transfer logic meets industry standards for accuracy and compliance— particularly the importance of "idempotency" (same request processed multiple times results in same outcome without duplicate charges), leading to implementation of unique transaction identifiers and atomic database sessions.

2.2 Existing Manual and Digital Tracking Systems

Target users rely on three primary approaches, each with critical limitations justifying unified solution development.

Paper-Based Tracking (~18% of participants): Manual recording in notebooks with calculator totals. Problems: data loss risk (no backups), no real-time insights (delayed awareness), no visualization (difficult pattern analysis), poor accessibility (location dependent).

Spreadsheet-Based Systems (47% of students): Custom Excel/Google Sheets templates with formulas. Problems: formula errors cascade throughout records, manual categorization becomes tedious, poor mobile usability discourages logging, no proactive budget alerts.

Commercial Apps: Bank account connections with automatic categorization. Problems: excessive automation reduces cognitive awareness, premium costs ($15-20/month), privacy concerns, complex OAuth authentication.

All methods are reactive, notifying users after problems occur; budgeting/goals/P2P remain fragmented across platforms; commercial features are cost-prohibitive while free versions prioritize monetization; excessive automation reduces involvement while manual methods become inconsistent (60% fall behind within 2-3 weeks). SFT addresses these by combining manual entry's cognitive benefits with modern analytical power while maintaining affordability for students.

A comprehensive comparative analysis of existing financial tracking approaches, identified gaps, and the SFT platform's contributions is provided in **Appendix B**.

[2.3 User Research: Financial Behaviors of Emerging

Adults]()

Quantitative research reveals a growing affordability crisis: 56% of undergraduates would struggle to secure $500 for unexpected expenses, and 68% had exhausted funds at least once during the year (NCAN, 2024)

| **Hardship Indicator** | **Statistic

(2024–2025)**	**Impact**	
Unexpected Expense Difficulty ($500)	56%	Increased financial vulnerability.
Exhausted Funds in Current Year	68%	Chronic stress affecting academic
focus.		
Basic Needs Insecurity	58%	Food and housing instability.
Skipping Meals to Save Money	67%	Deterioration in physical and mental
health.		
Daily Financial Worry	35%	Reduced cognitive bandwidth.

Table 3: Student Financial Hardship Statistics (2024-2025)

A significant "literacy-behavioral gap" exists: while 80% of students feel confident about money management, only 21% use budgeting applications, 80% have never created a formal budget, and 77% lack emergency funds (OECD, 2024) ; (Santander UK, 2025)This disparity suggests overconfidence may mask a critical lack of practical financial experience.

Digital platforms like TikTok have become the primary financial education source for 25 % of young adults, often providing fragmented advice focused on high-risk investments rather than fundamental cash flow management and emergency preparedness (Santander UK, 2025). The SFT platform counters this by emphasizing essential habits: tracking daily expenses and building liquid emergency funds.

A detailed questionnaire used to assess financial behaviors, literacy levels, and spending patterns among emerging adults is provided in **Appendix C**.

[2.4 Competitive Analysis of the Personal Finance

Ecosystem]()

The personal finance management (PFM) market in 2025 is dominated by several key players utilizing different methodologies: YNAB (zero-based budgeting, $109/year), Monarch Money (collaborative budgeting, no free tier), PocketGuard (snapshot approach), and Credit Karma (free but product-focused). Table 4 below provides a detailed feature comparison.

| **Feature** | **SFT Platform** | **YNAB** | **PocketGuard** | **Credit **Karma**

Budget Model	Manual-first	Zero-Based	Snapshot	Automated
Cost	Free/Open Source	$14.99/mo	$12.99/mo	Free
Predictive Insights	Linear Regression	None	Basic	None
P2P Transfer	Internal Wallet	None	None	None
Goal Tracking	Milestone-Based	Advanced	Basic	Basic
Mobile UX	Native MERN	Premium	Premium	Ad-heavy

Table 4: Competitive Feature Matrix: SFT vs. Industry Leaders

2.5 Identified Gaps and Behavioral Opportunities

The primary limitation of existing commercial systems is their reactive nature, notifying users of financial issues only after they occur and preventing proactive management (Deloitte, 2023). Research indicates that high levels of automation contribute to

"financial disengagement," where users become passive observers and lose awareness of their actual spending limits (Federal Reserve , 2024).

| **Pain Point** | **Behavioral Impact** | **SFT

Solution**		
Excessive Automation	Reduced expenditure awareness.	Structured manual entry.
Reactive Alerts	Behavioural correction occurs too	
late.	80/90/100% threshold alerts.	
Fragmented UX	Disconnected financial data.	Unified wallet and budgeting.
Lack of Forecasting	No forward-looking insights.	Linear regression forecasting.

Table 5: Pain Point Analysis of Automated Budgeting Tools

Users often rely on multiple applications for budgeting, P2P transfers, and savings, resulting in a fragmented experience that increases cognitive load (Deloitte, 2023).

SFT addresses these gaps by consolidating disparate functions into a unified ecosystem with an integrated P2P wallet that captures the "social" side of spending often difficult to track in traditional apps.

Chapter 03: Requirements Specification

Requirements are divided into functional and non-functional categories, refined during prototyping for software engineering alignment (Sommerville, 2021).

3.1 Functional Requirements

Functional requirements specify the essential capabilities and services that the system must provide to meet user needs and business objectives (Sommerville, 2021). Each requirement is uniquely identified using the FR-XX format for traceability throughout the development lifecycle.

FR-01: User Authentication and Authorization

The system shall provide secure user authentication through email and password credentials, implementing JWT-based stateless session management with role-based access control supporting four distinct user roles: Super Admin, Admin, User, and Guest.

FR-02: Transaction Management

The system shall enable users to create, view, update, and delete income and expense transactions with mandatory fields including amount, category, date, and optional description, supporting a minimum of 18 predefined expenditure categories.

FR-03: Budget Configuration and Monitoring

The system shall allow users to set category-specific monthly budget limits and automatically track expenditure against allocated budgets, calculating utilization percentages in real-time.

FR-04: Progressive Budget Alerts

The system shall automatically generate notifications when budget utilization reaches 80%, 90%, and 100% thresholds for any category, delivering alerts via in-app notifications and email.

FR-05: Savings Goal Tracking

The system shall enable users to create milestone-based savings goals with target amounts and deadline dates, displaying progress visualization through percentage completion indicators.

FR-06: Bill Management

The system shall support recurring bill registration with configurable frequency (monthly, quarterly, annually), due date tracking, and automated email reminders sent three days prior to payment deadlines.

FR-07: Peer-to-Peer (P2P) Wallet Transfers

The system shall facilitate internal peer-to-peer fund transfers between registered users through a secure wallet system implementing atomic transactions with complete audit trail logging.

FR-08: Financial Dashboard Visualization

The system shall present real-time financial summaries including total income, total expenses, current balance, category-wise spending breakdowns, and budget performance comparisons through interactive charts and graphs.

FR-09: Expense Forecasting

The system shall analyze historical transaction data using linear regression algorithms to predict future monthly expenditure trends, displaying forecast accuracy through confidence score indicators (R^2 values).

FR-10: Data Export and Reporting

The system shall provide export functionality for transaction histories and financial reports in CSV and PDF formats, filtered by customizable date ranges and categories.

The transaction management module serves as the core engine, supporting 18+ expenditure categories including food, transport, and education (Deloitte, 2023). These categories are linked to the budgeting and forecasting engines. To ensure data integrity, the P2P transfer system implements double-entry accounting principles, guaranteeing that every debit from one user's internal wallet corresponds to an atomic and equal credit in the recipient's wallet (Taduka, 2024). This is managed through MongoDB's multi-document ACID transactions, which prevent the "lost update" problem where funds could vanish if a server fails mid-transfer.

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that ensure the system operates reliably, securely, and efficiently within the intended operational environment (Sommerville, 2021).

NFR-01: Performance

The system shall maintain API response times below 300 milliseconds (P95 percentile) for all standard CRUD operations and load dashboard pages within 3 seconds on standard broadband connections (10 Mbps).

NFR-02: Security

The system shall implement industry-standard security measures including bcrypt password hashing with 10 salt rounds, HTTPS/TLS encryption for all data transmission, and HTTP-only secure cookies for JWT token storage.

NFR-03: Usability

The system shall adhere to WCAG 2.1 Level AA accessibility standards, ensuring keyboard navigation support, minimum 4.5:1 color contrast ratios, and responsive design compatible with desktop browsers (Chrome, Firefox, Safari, Edge) and mobile devices.

NFR-04: Reliability

The system shall implement MongoDB ACID transactions for all critical financial operations (P2P transfers, budget updates) to ensure data consistency and implement comprehensive error handling with user-friendly error messages.

3.3 Software Requirements

The following software components constitute the development and production technology stack:

Backend Technologies:

- Node.js (v18.17.0 or higher) - JavaScript runtime environment
- Express.js (v5.2.1) - Web application framework
- MongoDB (v9.1.1) - NoSQL database system
- Mongoose (v8.0) - Object Data Modeling (ODM) library

Frontend Technologies:

- React (v19.2.0) - JavaScript UI library
- Vite (v7.2.4) - Build tool and development server
- Tailwind CSS (v3.4.17) - Utility-first CSS framework
- Recharts - Data visualization library
- Axios - HTTP client for API communication

Development Tools:

- Visual Studio Code (v1.85 or higher) - Integrated development environment

- Git (v2.40+) - Version control system
- GitHub - Repository hosting and collaboration platform
- Docker Desktop (v4.25) - Containerization platform
- MongoDB Compass (v1.40) - Database GUI management tool

Testing Frameworks:

- Jest (v29.7) - Backend unit and integration testing
- Vitest (v1.0) - Frontend component testing
- Supertest (v6.3) - HTTP API endpoint testing

Third-Party Services:

- MongoDB Atlas - Cloud database hosting (M0 Free Tier)
- Render - Backend API deployment platform
- Vercel - Frontend static hosting platform
- SendGrid API - Transactional email service
- Fixer.io API - Currency exchange rate data

3.4 Hardware Requirements

The following hardware specifications define the minimum and recommended configurations for development and deployment environments:

Development Environment:

- **Processor:** Dual-core CPU @ 2.0 GHz (minimum); Quad-core @ 2.5 GHz (recommended)
- **RAM:** 8 GB (minimum); 16 GB (recommended)
- **Storage:** 256 GB SSD with at least 10 GB free space
- **Network:** Broadband internet connection (10 Mbps minimum)
- **Display:** 1920×1080 resolution (minimum)

Production/Deployment Environment:

- **Server:** Cloud-based virtual machine (VM) or container instance
- **CPU:** 1-2 vCPUs (MongoDB Atlas M0 / Render Free Tier)
- **RAM:** 512 MB - 1 GB (shared hosting environment)
- **Storage:** 512 MB - 1 GB persistent storage (MongoDB Atlas)
- **Network:** High-availability internet connection with 99.9% uptime

- **Bandwidth:** Minimum 100 GB monthly data transfer

Client (End User) Requirements:

- **Device:** Desktop computer, laptop, tablet, or smartphone
- **Processor:** Any modern processor capable of running current web browsers
- **RAM:** 4 GB minimum
- **Browser:** Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- **Network:** Stable internet connection (5 Mbps minimum for optimal performance)
- **Screen Resolution:** 1366×768 minimum; responsive design supports 320px minimum width

3.5 Accessibility Standards: WCAG 2.1 AA Compliance

Accessibility ensures inclusivity and legal compliance in financial services (Siteimprove, n.d.).

Criteria	Standard	Implementation Strategy
Colour Contrast	Success Criterion 1.4.3	Minimum 4.5:1 contrast ratio
Keyboard Navigation	Success Criterion 2.1.1	No keyboard traps; visible focus
states		
Alt Text	Success Criterion 1.1.1	Descriptive alternatives for charts
and icons		
Text Scaling	Success Criterion 1.4.4	Resizable up to 200% without layout
breakage		
Error Feedback	Success Criterion 3.3.2	Descriptive validation messaging

Table 8: WCAG 2.1 AA Accessibility Success Criteria for Dashboards

SFT implements screen reader testing with ARIA-labels for visualizations ensuring logical tab order (Elementor, n.d.).

[Chapter 04: Feasibility Study and Strategic

Assessment]()

Feasibility study validates SFT viability within academic constraints while maintaining industrial standards (Pressman & Maxim, 2020).

4.1 Operational and Technical Feasibility

Mobile-first design enables on-the-go recording critical for adoption (Dey, 2026). Fullstack JavaScript mastery reduces complexity, enabling code sharing between client/server (Enfin Technologies, n.d.). Node.js handles asynchronous processing; React's virtual DOM ensures real-time updates (Suryavanshi, 2024).

[4.2 Economic Feasibility: Comprehensive

Cost-Benefit Analysis]()

MERN utilizes free, open-source tools eliminating licensing costs (Bitcot, 2026).

Infrastructure Component	Service Provider	Plan / Monthly Cost	Efficiency Gain
Database	MongoDB Atlas (M10)	~$66.82/mo	Managed performance and scaling
Hosting (Frontend/API)	AWS Amplify	~$8.08+	Elastic scalability
CI/CD Pipeline	CircleCI Performance	~$15.00/mo	Automated test deployment

Infrastructure Component	Service Provider	Plan / Monthly Cost	Efficiency Gain
Email API	SendGrid Essentials	~$19.95/mo	Reliable transactional alerts
Currency API	Fixer.io Basic	~$14.99/mo	Accurate midpoint exchange rates

Table 9: Production Environment Infrastructure Cost Projection

Free tiers from MongoDB Atlas, Render, and GitHub Actions enable zero-cost prototyping.

4.3 Risk Management and Mitigation Strategies

Financial platforms address technical/security risks through proactive mitigation

(Taduka, 2024).

Risk Description	Severity	Impact	Mitigation Strategy
Data Atomicity Failure	Critical	Loss of funds during transfer	MongoDB ACID sessions
JWT Token Hijacking	High	Unauthorized access	HTTP-only Secure cookies
API Rate Limiting	Medium	Dashboard delays	Redis caching layer
Low Data Accuracy	Medium	Poor forecasting reliability	Minimum dataset thresholds

Table 10: Project Risk Assessment and Impact Matrix

Critical risks mitigated through MongoDB replica set transactions and HttpOnly cookies with helmet middleware. (MongoDB Inc., 2024)

4.4 Schedule Feasibility and Timeline Practicality

Schedule feasibility evaluates realistic completion within 20 weeks (November 14, 2025; April 2, 2026) through 10 two-week Agile sprints.

Timeline Breakdown

| **Phase** | **Sprints** | **Duration** | **Focus Areas** | **Feasibility

Rationale**			
Foundation	1–2	4 weeks	Authentication,
CRUD, Dashboard	Moderate		
complexity			
Core Features	3–5	6 weeks	Budgets,
Goals, Bills	Business		
logic manageable			
Advanced			
Features	6–8	6 weeks	P2P Wallet,
Analytics	ACID learning		
curve allocated			
Enhancement	9–10	4 weeks	Multi-currency,
Testing	Buffer sprint		
included			

Table 11 Project Risk Assessment and Impact Matrix

Feasibility Factors.

JavaScript Full-Stack: Eliminates context-switching, increasing productive time by 1520%. Agile Sprint Buffer: Sprint 10 serves as explicit buffer. Free-Tier Infrastructure: Enables provisioning in minutes. Component Reusability: React architecture enables significant code reuse.

Key Risks and Mitigation

MongoDB ACID Transactions: P2P requires multi-document atomicity. *Mitigation:*

Allocated 2 sprints with initial focus on single-document operations. Render Cold Starts: 8-15 second delays. *Mitigation:* Keep-alive ping service; paid tier option. Scope Creep: *Mitigation:* Strict MoSCoW prioritization.

Validation of Feasibility

As of March 2, 2026, Sprints 1-8 are 90%+ complete; only Sprints 9-10 remain. All "Must Have" features are implemented and tested with no sprint requiring >1 week spillover.

Chapter 05: System Architecture

5.1 Use Case Diagram

The following use case diagram illustrates the functional requirements and actor interactions within the SFT platform using standard UML notation. The diagram employs stick figures for human actors, system boundary rectangles, and proper relationship notations including generalization (inheritance), <>, and <> dependencies.

```
@startuml
left to right direction
skinparam packageStyle rectangle

' Define actors using stick figures (default UML notation)
actor "Guest User" as Guest
actor "User" as User
actor "Admin" as Admin
actor "Super Admin" as SuperAdmin

' External system actors
actor "Email System\n(SendGrid API)" as EmailSystem
actor "Currency API\n(Fixer.io)" as CurrencyAPI

' System boundary
rectangle "Smart Financial Manager (SFT) Platform" {

  ' Authentication use cases
  usecase "User Authentication" as UC01
  usecase "Validate Credentials" as UC01a
  usecase "Generate JWT Token" as UC01b
```

```
' Transaction management
usecase "Manage Transactions" as UC02
usecase "Create Transaction" as UC02a
usecase "View Transactions" as UC02b
usecase "Update Transaction" as UC02c
usecase "Delete Transaction" as UC02d
usecase "Filter by Category" as UC02e

' Budget management
usecase "Manage Budget" as UC03
usecase "Set Budget Limits" as UC03a
usecase "Track Budget Utilization" as UC03b
usecase "Generate Budget Alerts" as UC03c
usecase "Send 80% Alert" as UC03c1
usecase "Send 90% Alert" as UC03c2
usecase "Send 100% Alert" as UC03c3

' Goals tracking
usecase "Track Savings Goals" as UC04
usecase "Create Goal" as UC04a
usecase "Update Goal Progress" as UC04b
usecase "View Goal Status" as UC04c

' Bill management
usecase "Manage Bills" as UC05
usecase "Register Recurring Bill" as UC05a
usecase "Track Due Dates" as UC05b
usecase "Send Bill Reminders" as UC05c
usecase "Mark Bill as Paid" as UC05d

' P2P wallet
usecase "Perform P2P Transfer" as UC06
usecase "Validate Wallet Balance" as UC06a
usecase "Execute ACID Transaction" as UC06b
usecase "Log Transfer Audit" as UC06c

' Dashboard and analytics
usecase "View Financial Dashboard" as UC07
usecase "Generate Financial Summary" as UC07a
usecase "Visualize Spending Patterns" as UC07b

' Forecasting
usecase "Generate Expense Forecast" as UC08
usecase "Apply Linear Regression" as UC08a
usecase "Calculate R² Score" as UC08b

' Reporting
usecase "Export Financial Reports" as UC09
usecase "Generate CSV Export" as UC09a
usecase "Generate PDF Export" as UC09b

' Admin functions
usecase "Manage Users" as UC10
usecase "View System Analytics" as UC11
usecase "Configure System Settings" as UC12
```

```
  ' Demo access
  usecase "View Demo Dashboard" as UC13
}

' Actor inheritance (generalization)
User <|-- Admin
Admin <|-- SuperAdmin

' Guest associations
Guest --> UC13

' User associations
User --> UC01
User --> UC02
User --> UC03
User --> UC04
User --> UC05
User --> UC06
User --> UC07
User --> UC08
User --> UC09

' Admin associations (inherits User + additional)
Admin --> UC10
Admin --> UC11

' Super Admin associations (inherits Admin + additional)
SuperAdmin --> UC12

' <<include>> relationships (mandatory sub-functionality)
UC01 ..> UC01a : <<include>>
UC01 ..> UC01b : <<include>>
UC02 ..> UC02b : <<include>>
UC03 ..> UC03b : <<include>>
UC06 ..> UC06a : <<include>>
UC06 ..> UC06b : <<include>>
UC06 ..> UC06c : <<include>>
UC07 ..> UC07a : <<include>>
UC08 ..> UC08a : <<include>>
UC08 ..> UC08b : <<include>>

' <<extend>> relationships (optional enhancement)
UC03c ..> UC03 : <<extend>>
UC03c1 ..> UC03c : <<extend>>
UC03c2 ..> UC03c : <<extend>>
UC03c3 ..> UC03c : <<extend>>
UC05c ..> UC05 : <<extend>>

' External system interactions
UC03c --> EmailSystem : sends notification
UC05c --> EmailSystem : sends reminder
UC02 --> CurrencyAPI : fetches exchange rates

@enduml
```

Figure 1: Use Case Diagram of the Smart Financial Manager (SFT) Platform

UML Notation Key:

- **Stick Figures:** Human actors (Guest User, User, Admin, Super Admin)
- **Rectangle:** System boundary containing all use cases
- **Ovals:** Use cases (system functionalities)
- **Solid Lines:** Associations between actors and use cases
- **Dashed Arrows with <>:** Mandatory sub-functionality always executed
- **Dashed Arrows with <>:** Optional functionality triggered conditionally
- **Solid Arrows (generalization):** Inheritance relationship between actors
- **External Actors:** Represented as stick figures outside the primary actor hierarchy

Use Case Diagram Assumptions

A1. Actor Hierarchy

· Admin role inherits all User capabilities, plus administrative functions.

· Super Admin inherits all Admin capabilities, plus system-wide configuration privileges.

· Guest users have read-only access to demo features without authentication.

A2. Authentication Model

· All registered users (User, Admin, Super Admin) must authenticate before accessing protected features.

· JWT-based stateless authentication is used, eliminating server-side session storage.

· Guest users can view the demo dashboard without login credentials.

A3. Use Case Relationships

· <<include>>: Mandatory sub-functionality (e.g., "Manage Transactions" always includes "View Transactions").

· <<extend>>: Optional enhancement (e.g., "Budget Alerts" extends "Manage Budget" only when threshold is reached).

· Generalization: Admin and Super Admin extend User capabilities through role inheritance.

A4. External System Dependencies

· Email System (SendGrid API) handles all outbound notifications, such as bill reminders and budget alerts.

· Currency API (Fixer.io) provides real-time exchange rates for multi-currency transactions.

A5. Security Boundaries

· Guest users cannot modify any data (read-only access).

· Users can only access their own financial data, enforced via userId filtering.

· Admins can view all user data but cannot modify other users' financial records.

· Super Admins have unrestricted system access.

A6. Business Rules

· P2P transfers require both sender and receiver to have active wallets.

· Budget alerts trigger automatically at 80%, 90%, and 100% thresholds.

· Financial analytics require a minimum of 30 days of transaction history.

[5.2 Class

Diagram of Proposed System]()

Figure 2: Class Diagram of Proposed System

Class Diagram Assumptions

A1. Inheritance Model

· `<span>Admin</span>` class inherits from `<span>User</span>`, gaining all user capabilities plus administrative methods.

· `<span>Super Admin</span>` (not shown separately) is treated as an `<span>Admin</span>` with an elevated permissions array.

A2. Visibility Modifiers

· **Private (-)** : All attributes are private to enforce encapsulation.

· **Public (+)** : All methods are public to provide controlled access interfaces.

A3. Multiplicity

· **1:1 relationships** : Each `<span>User</span>` owns exactly one `<span>Wallet</span>` and has one `<span>FinancialHealthScore</span>`.

· **1:Many relationships** : One `<span>User</span>` can have many `<span>Transactions</span>`, `<span>Budgets</span>`, `<span>Goals</span>`, and `<span>Bills</span>`.

· **Many:1 relationships** : Many `<span>Transactions</span>` belong to one `<span>Category</span>`.

A4. Association Types

· **Composition (filled diamond)** : Strong ownership (e.g., `<span>User</span>` owns `<span>Wallet</span>`; if `<span>User</span>` is deleted, the `<span>Wallet</span>` is deleted).

· **Aggregation (hollow diamond)** : Weak ownership (e.g., `<span>Budget</span>` applies to `<span>Category</span>`; `<span>Category</span>` exists independently).

· **Dependency (dashed arrow)** : Temporary relationship (e.g., `<span>Budget</span>` triggers `<span>Notification</span>` only when threshold is reached).

A5. Data Types

· **ObjectId** : MongoDB's unique identifier (indexed primary key).

· **Decimal** : Precision financial amounts (using `<span>Decimal128</span>` in MongoDB).

· **Enum** : Restricted values (e.g., `<span>role: ['User', 'Admin', 'SuperAdmin']</span>`, `<span>type: ['Income', 'Expense']</span>`).

· **Date** : ISO 8601 timestamp format.

A6. Method Behavior

· **CRUD operations** : `<span>create()</span>`, `<span>update()</span>`, `<span>delete()</span>`, `<span>get()</span>` follow RESTful principles.

· **Validation methods** : `<span>validate*()</span>` throw exceptions on failure, return `<span>false</span>` on validation errors.

· **Calculation methods** : `<span>calculate*()</span>` perform real-time computations without database persistence.

A7. Business Logic & Security: `Wallet.transfer()` implements ACID transactions; `Budget.checkThreshold()` triggers alerts at 80/90/100%; passwords stored as bcrypt hash; P2PTransfer uses UUID v4 for idempotency; Admin permissions array-based.

A8. Performance & Scalability: Category pre-loaded as singleton; FinancialHealthScore cached in Redis 24h; stateless services with `userId` parameter for multi-tenant isolation; aggregation pipeline pushdown to MongoDB.

[5.3 High-Level

Architectural Diagram]()

Architectural Assumptions

· **Stateless Backend & SPA**: JWT-based stateless authentication enables horizontal scalability; React SPA with client-side routing reduces server load.

· **RESTful API & Caching**: Standard HTTP methods with JSON payloads; Redis caching for frequently accessed data (currency rates, tokens) with TTL configurations.

· **External Service Integration** :

o Third-party APIs (e.g., Fixer.io for currency rates, SendGrid for email notifications) are abstracted through a service layer.

o Facilitates easy replacement of providers if needed.

· **Database Design** :

o MongoDB’s document-based structure allows flexible schema evolution without migrations.

o ACID transaction support is used for critical financial operations such as P2P transfers.

· **Deployment Model** :

o Frontend hosted on static hosting (AWS Amplify / Vercel).

o Backend on Node.js runtime (Render / Railway).

o Database on managed MongoDB Atlas.

o Each tier can scale independently.

· **Security Boundaries** :

o All tiers communicate over HTTPS/TLS.

o Environment-specific configurations (`<span>.env</span>`) keep sensitive credentials isolated from the codebase.

· **Monitoring & Logging** :

o Application logs critical events (e.g., failed transactions, authentication attempts, API errors).

o Supports audit trails and debugging purposes.

5.4 Networking Diagram![]
(file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image012.jpg)

Figure 4: Networking Diagram

[Chapter 06: Development Tools and Technology

Justification]()

Technology selection was driven by the requirement for a unified ecosystem capable of supporting rapid iteration and complex financial logic (Enfin Technologies, n.d.).

6.1 Development Methodology: Agile Scrum Framework

The SFT platform adopts the Agile Scrum methodology to support iterative development, enabling rapid prototyping and continuous stakeholder feedback across 10 structured sprints (October 2025–March 2026). Each two-week sprint delivers functional increments while accommodating evolving requirements and refinement cycles (Schwaber & Sutherland, 2020).

Sprint Breakdown and Progress

| **Sprint** | **Primary Deliverable** | **Completion **Status**

Sprint 1	Project Setup & Authentication	100%
Sprint 2	Transaction CRUD	100%
Sprint 3	Budget Module	95%
Sprint 4	Goal Tracking	95%
Sprint 5	Bill Management	90%
Sprint 6	P2P Wallet	85%
Sprint 7	Analytics Dashboard	85%
Sprint 8	Multi-Currency	80%
Sprint 9	Testing & Refinement	In Progress
Sprint 10	Buffer & Documentation	Pending

Table 12: Sprint Progress Tracker

Backlog Prioritization

The MoSCoW prioritisation framework was implemented to distinguish:

· **Must-Have** : Authentication, CRUD operations, budget management

· **Should-Have** : P2P wallet, analytics dashboard

· **Could-Have** : Multi-currency support, AI-based enhancements

This structured prioritisation ensured that critical financial functionality was stabilised before introducing advanced features.

Definition of Done

A feature was considered complete only when: (1) code meets standards with 80%+ test coverage, (2) cross-browser compatibility validated, and (3) supervisor approval obtained.

Agile Flexibility

User research revealed that 34% of participants manage multi-currency expenses. In response, the Fixer.io API integration was incorporated into Sprint 9 without disrupting previously completed work. This demonstrates the adaptive capability of the Agile framework.

Tools

· GitHub Projects (Kanban-based tracking)

· Git (GitFlow branching strategy)

· GitHub Actions (CI/CD pipeline automation)

· Jest and Vitest (unit and integration testing)

· Markdown-based technical documentation

[6.2 The MERN Stack: An

Architectural Defense]()

The MERN stack’s full-stack JavaScript architecture enables significantly faster prototyping compared to Django or Spring Boot due to unified language usage across frontend and backend environments (Enfin Technologies, n.d.) .

| **Framework** | **Language** | **Scalability** | **Learning Curve** | **Selection

Rationale**				
Express.js	JavaScript	High	Moderate	Unified stack compatibility
Django	Python	High	Steep	Separate language ecosystem
Spring Boot	Java	Very High	High	Enterprise-oriented complexity
Ruby on Rails	Ruby	Moderate	Moderate	Rapid prototyping (declining
enterprise adoption)				

Table 13: Comparative Analysis of Backend Frameworks (2025)

Although Spring Boot dominates enterprise banking infrastructure, the MERN stack excels in real-time, interactive, user-centric applications requiring rapid UI updates and iterative refinement (Bitcot, 2026).

[6.3 Frontend Architecture and

Rendering Strategy]()

React's component-based architecture, combined with its Virtual DOM implementation, enables responsive real-time dashboard updates essential for financial data visualisation (Deloitte, 2023).

Tool	Purpose	Rationale
Vite	Build Tool	Significantly faster development
start-up time compared to Webpack		
Tailwind CSS	UI Framework	Utility-first architecture
accelerates UI development		
Recharts	Data Visualisation	Composable React components optimised

Tool	Purpose	Rationale
for financial charts		
Lucide React	Icon Library	Clean, scalable icons supporting 18+
spending categories		

Table 14: Frontend Framework Selection Rationale

6.4 Backend and Persistence Layers

Node.js employs an asynchronous, event-driven architecture capable of efficiently handling I/O-intensive financial operations. MongoDB's BSON document model maps directly to JavaScript objects, enabling seamless data handling across the stack (SourceForge, n.d.); (CSAC, 2025); (Suryavanshi, 2024). This architectural alignment reduces serialization overhead and enhances development efficiency.

[6.5 Algorithmic Foundations of the

SFT Platform]()

The intelligence layer of the SFT platform comprises four core algorithms that transform raw transactional data into actionable financial insights (Deloitte, 2023). Each algorithm addresses specific user needs identified during empirical research interviews.

6.5.1 Expense Forecasting: Linear Regression

The forecasting engine implements Simple Linear Regression to predict future monthly expenses based on historical trends. Using the least squares method, the system calculates the trend line:

$$y = mx + b$$

Where:

$m = (n\sum xy - \sum x\sum y) / (n\sum x^2 - (\sum x)^2)$ represents the rate of spending change

$b = (\sum y - m\sum x) / n$ represents baseline spending

The algorithm requires a minimum of three months of historical data and returns predictions with R^2 confidence scores:

· 0.7 = High accuracy

· 0.4–0.7 = Medium accuracy

· <0.4 = Low accuracy

Forecasts are suppressed when insufficient data exists to prevent misleading outputs. This mechanism enables users to anticipate potential financial shortfalls weeks in advance.

[6.5.2 Anomaly Detection: Statistical Z-Score

Analysis]()

To automatically identify unusual spending behaviour, the system calculates Z-scores for monthly total expenses:

$Z = (X - \mu) / \sigma$

Where:

X = Current month’s expense

μ = Mean expense

σ = Standard deviation

Any month with $|Z| \geq 2$ (representing approximately the top or bottom 5% of a normal distribution) is flagged as statistically anomalous:

· $Z \geq 2 \rightarrow$ High Spending

· $Z \leq -2 \rightarrow$ Low Spending

The anomaly detector analyses rolling six-month windows and requires a minimum of three months of data. This approach surfaces hidden financial irregularities without

requiring manual review.

[6.5.3 Financial Health Score: Multi-Factor Weighted

Algorithm]()

The Financial Health Score synthesises five behavioural dimensions into a single 0–100 metric using the weighted formula:

Health Score =

(30% × Budget Adherence) +

(25% × Savings Rate) +

(20% × Emergency Fund) +

(15% × Transaction Consistency) +

(10% × Debt-to-Income Ratio)

Scoring Logic:

· Budget adherence awards 100 for under-budget performance, decreasing proportionally with overspending.

· Savings rate assigns full marks for ≥20% savings rate.

· Emergency fund calculated relative to $1,000 benchmark.

The weighting prioritises budgeting and saving behaviours as primary determinants of financial wellness.

6.5.4 Loan EMI Calculator: Banking-Standard Formula

The loan management module implements the industry-standard Equated Monthly Installment (EMI) formula:

$EMI = [P \times r \times (1 + r)^n] / [(1 + r)^n - 1]$

Where:

P = Principal amount

r = Monthly interest rate (annual rate ÷ 12 ÷ 100)

n = Loan tenure in months

The calculator generates full amortisation schedules and computes total interest payable, aligning with banking standards for loan comparison.

These algorithms collectively demonstrate the practical application of statistical modelling, predictive analytics, and financial mathematics to address real-world user challenges identified through empirical research.

[Chapter 07:

Implementation Progress]()

7.1 Development Environment Setup

The development environment was configured to support full-stack MERN development using industry-standard tools, structured workflows, and version control best practices.

IDE and Code Editor

- **Visual Studio Code (v1.85)** with extensions: ESLint, Prettier, MongoDB for VS Code, GitLens, Thunder Client (API testing)
- **Node.js v18.17.0** as the JavaScript runtime environment
- **npm v9.8.1** for dependency and package management

Version Control System

- **GitHub** repository using GitFlow branching strategy (main, develop, feature/*, hotfix/*)
- Branch protection rules enforcing pull request reviews before merging into main
- Commit conventions following the Conventional Commits specification (feat:, fix:, docs:, refactor:)

Containerization and Deployment

- **Docker Desktop v4.25** for local MongoDB containerization
- **Docker Compose** orchestrating Node.js backend and MongoDB services
- **Render** for backend API deployment (Node.js runtime environment)
- **Vercel** for React frontend deployment (static hosting)

Database Management

- **MongoDB Atlas M0 (Free Tier)** cluster for cloud database hosting
- **MongoDB Compass v1.40** for visual database management and query debugging
- **Mongoose v8.0** ODM for schema modeling and validation

Testing Framework

- **Jest v29.7** for backend unit and integration testing
- **Vitest v1.0** for frontend component testing
- **Supertest v6.3** for API endpoint testing

CI/CD Pipeline

- **GitHub Actions** automating test execution on every push
- **npm scripts** for build, test, and deployment workflows
- Environment variables managed via .env files (development) and platform secrets (production)

This modular and automated environment supports rapid iteration while maintaining code quality through structured testing and consistent deployment practices.

[7.2

Implemented Features]()

As of March 2, 2026, the following features have been implemented and tested:

Authentication & Authorization (100% Complete)

- User registration with email validation and password strength enforcement
- JWT-based stateless authentication using HTTP-only cookies

- Role-Based Access Control (RBAC) supporting four roles: Super Admin, Admin, User, Guest
- Password reset functionality using email verification tokens

Transaction Management (100% Complete)

- Full CRUD operations for income and expense transactions
- Support for 18+ transaction categories (Food, Transport, Education, Entertainment, etc.)
- Date-range filtering and category-based retrieval
- Pagination reducing API payload sizes from 500KB+ to under 50KB

Budget Management (95% Complete)

- Category-specific budget configuration
- Real-time utilization tracking with automatic spent amount updates
- Threshold alerts at 80%, 90%, and 100%
- Monthly automatic reset functionality

Goal Tracking (95% Complete)

- Milestone-based savings goal creation with target amounts and dates
- Percentage-based progress visualization
- Contribution tracking with automatic status updates
- Goal completion notifications

Bill Management (90% Complete)

- Recurring bill registration (Monthly, Quarterly, Annually)
- Due date tracking and upcoming bill visibility
- Mark-as-paid functionality
- Email reminders sent three days before due dates using node-cron

P2P Wallet System (85% Complete)

- Internal wallet with balance tracking
- Peer-to-peer transfers using MongoDB ACID transactions
- Atomic debit/credit operations with rollback support
- Transfer history with audit trail logging

Analytics Dashboard (85% Complete)

- Real-time financial summary cards (Income, Expenses, Balance)
- Category-wise spending breakdown using Recharts
- Monthly expense trend visualization
- Budget vs. Actual comparison charts

Financial Health Score (40% Complete)

- Multi-factor weighted algorithm (0–100 score)
- Five scoring dimensions:
  - Budget Adherence (30%)
  - Savings Rate (25%)
  - Emergency Fund (20%)
  - Consistency (15%)
  - Debt Ratio (10%)
- Monthly recalculation with historical tracking

Dark Mode Support (80% Complete)

- System-wide theme toggle (Light/Dark)
- localStorage persistence to prevent flash of unstyled content
- WCAG 2.1 AA contrast-compliant color schemes

Expense Forecasting (40% Complete)

- Linear regression algorithm predicting future expenses
- R^2 confidence score reporting (High >0.7, Medium 0.4–0.7, Low <0.4)
- Minimum three-month data requirement

[7.3

Screenshots and Code Snippets]()

This section provides implementation evidence through representative backend and frontend code samples.

7.3.1 Backend Code Snippets

JWT Authentication Middleware

// backend/middleware/authMiddleware.js

const jwt = require('jsonwebtoken');

const User = require('../models/User');

// Protect routes requiring authentication

exports.protect = async (req, res, next) => {

try {

```
const token = req.cookies.jwtToken;

if (!token) {
```

return res.status(401).json({

```
success: false,

message: 'Not authorized, no
```

token provided'

```
});

}

// Verify token

const decoded = jwt.verify(token,
```

process.env.JWT_SECRET);

```
// Attach user to request object
```

(excluding password)

```
req.user = await
```

User.findById(decoded.id).select('-password');

```
if (!req.user) {

return res.status(401).json({

success: false,

message: 'User not found'

});

}

next();
```

} catch (error) {

```
return res.status(401).json({

success: false,

message: 'Token verification
```

failed'

```
});
```

}

};

// Restrict access based on user roles

exports.authorize = (...roles) => {

return (req, res, next) => {

```
if (!roles.includes(req.user.role)) {

return res.status(403).json({

success: false,
```

```
message:`Role '${req.user.role}' is not authorized to access this resource`

});

}

next();
```

};

};

Budget Alert Logic

// backend/controllers/budgetController.js

const Budget = require('../models/Budget');

const Transaction = require('../models/Transaction');

const { sendNotification } = require('../services/notificationService');

// Check budget threshold and send alerts

const checkBudgetThreshold = async (userId, category) => {

try {

```
// Retrieve budget for category

const budget = await Budget.findOne({
```

userId, category });

```
if (!budget) return;

// Aggregate total expenses for
```

category

```
const expenses = await
```

Transaction.aggregate([

```
{
$match: {
userId:
```

mongoose.Types.ObjectId(userId),

```
category,
type: 'Expense'
}
},
{
$group: {
_id: null,
total: {$sum: '$amount' }
}
}
]);
const totalSpent = expenses[0]?.total
```

|| 0;

```
const utilization = (totalSpent /
```

budget.limitAmount) * 100;

```
// Update spent amount in budget
budget.spentAmount = totalSpent;
await budget.save();
```

```
// Send threshold-based notifications

if (utilization >= 100 &&
```

!budget.alert100Sent) {

```
await sendNotification(userId, {

title:`${category} Budget EXCEEDED!`,

message: `You have spent
```

{totalSpent} of your $${budget.limitAmount} ${category} budget.`, type: 'budget_alert' }); budget.alert100Sent = true; } else if (utilization >= 90 && !budget.alert90Sent) { await sendNotification(userId, { title: `${category} Budget Warning`, message: `You have used 90% of your ${category} budget. Only $${budget.limitAmount - totalSpent} remaining.`, type: 'budget_warning' }); budget.alert90Sent = true; } else if (utilization >= 80 && !budget.alert80Sent) { await sendNotification(userId, { title: `${category} Budget Notice`, message: `You have used 80% of your ${category} budget.`, type: 'budget_notice' }); budget.alert80Sent = true; } await budget.save(); } catch (error) { console.error('Budget threshold check error:', error); } }; module.exports = { checkBudgetThreshold }; P2P Transfer with ACID Transactions // backend/controllers/transferController.js const mongoose = require('mongoose'); const Wallet = require('../models/Wallet'); const P2PTransfer = require('../models/P2PTransfer'); const { v4: uuidv4 } = require('uuid'); // Execute atomic P2P transfer exports.executeTransfer = async (req, res) => { const session = await mongoose.startSession(); session.startTransaction(); try { const { receiverId, amount, description } = req.body; const senderId = req.user._id; // Validate transfer amount if (amount <= 0) { throw new Error('Transfer amount must be positive'); } // Retrieve sender wallet const senderWallet = await Wallet.findOne({ userId: senderId }).session(session); if (!senderWallet) { throw new Error('Sender wallet not found'); } if (senderWallet.balance < amount) { throw new Error('Insufficient funds'); } // Retrieve receiver wallet const receiverWallet = await Wallet.findOne({ userId: receiverId }).session(session); if (!receiverWallet) { throw new Error('Receiver wallet not found'); } // Debit sender wallet senderWallet.balance -= amount; senderWallet.lastUpdated = Date.now(); await senderWallet.save({ session }); // Credit receiver wallet receiverWallet.balance += amount; receiverWallet.lastUpdated = Date.now(); await receiverWallet.save({ session }); // Create transfer record with unique transaction ID

(idempotency) const transfer = await P2PTransfer.create([{ senderId, receiverId, amount, description, status: 'Completed', transactionId: uuidv4() }], { session }); // Commit transaction await session.commitTransaction(); res.status(200).json({ success: true, message: 'Transfer completed successfully', transfer: transfer[0] }); } catch (error) { // Rollback on any error await session.abortTransaction(); res.status(500).json({ success: false, message: error.message || 'Transfer failed' }); } finally { session.endSession(); } }; ``` ### [7.3.2 Frontend Code Snippets]() ****Dashboard Component with Recharts**** The following component implements the financial dashboard using **React** functional components and hooks. Data visualization is handled using **Recharts**, while HTTP requests are performed via **Axios**. The component retrieves summary statistics and budget data from backend API endpoints, transforms the dataset for chart compatibility, and renders both summary cards and a budget performance bar chart. ``` // frontend/src/components/Dashboard.jsx import React, { useState, useEffect } from 'react'; import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts'; import axios from 'axios'; const Dashboard = () => { const [financialData, setFinancialData] = useState({   totalIncome: 0,   totalExpenses: 0,   currentBalance: 0 }); const [budgets, setBudgets] = useState([]); const [loading, setLoading] = useState(true); useEffect(() => {   fetchDashboardData(); }, []);   const fetchDashboardData = async () => {   try {   const [summaryRes, budgetRes] = await Promise.all([axios.get('/api/transactions/summary'),   axios.get('/api/budgets')   ]); setFinancialData(summaryRes.data.data);   setBudgets(budgetRes.data.budgets);   setLoading(false);   } catch (error) {   console.error('Dashboard data fetch error:', error);   setLoading(false);   } };   // Transform budget data for chart visualization const chartData = budgets.map(budget => ({   category: budget.category,   budget: budget.limitAmount,   spent: budget.spentAmount, remaining: budget.limitAmount - budget.spentAmount }));   if (loading) {   return Loading... ; }   return (<h1 className="text-3xl font-bold mb-6 text-gray-800 dark:text-white">   Financial Dashboard   </h1>   {/* Summary Cards */}   <h3 className="text-sm text-gray-500 dark:text-gray-400">Total Income</h3> ${financialData.totalIncome}</p>   </div>   <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">   <h3 className="text-sm text-gray-500 dark:text-gray-400">Total Expenses</h3>   <p className="text-2xl font-bold text-red-600">${financialData.totalExpenses}   <div

className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow"> <h3 className="text-sm text-gray-500 dark:text-gray-400">Current Balance</h3> ${financialData.currentBalance}   {/* Budget vs Spent Chart */}   <h2 className="text-xl font-semibold mb-4 text-gray-800 dark:text-white"> Budget Performance </h2> <ResponsiveContainer width="100%" height={400}> <BarChart data={chartData}> <CartesianGrid strokeDasharray="3 3" /> <XAxis dataKey="category" /> <YAxis /> <Tooltip /> <Legend /> <Bar dataKey="spent" fill="#EF4444" name="Spent" /> <Bar dataKey="remaining" fill="#10B981" name="Remaining" /> </BarChart> </ResponsiveContainer>   ); }; export default Dashboard; ``` This implementation demonstrates: · Concurrent API data fetching using `<span>Promise.all</span>` · State management via React Hooks (`<span>useState</span>`, `<span>useEffect</span>`) · Client-side data transformation for visualization · Responsive chart rendering · Integrated dark mode styling through conditional CSS classes ****Dark Mode Implementation**** The application-level theme toggle is implemented within the root component using React state management and `<span>localStorage</span>` persistence. Client-side routing is handled by **React Router**. On initial mount, the application checks for a previously saved theme preference and applies the appropriate CSS class to the root `<span><html></span>` element. The toggle function updates both the DOM class list and `<span>localStorage</span>` to ensure persistence across sessions. ``` // frontend/src/App.jsx import React, { useState, useEffect } from 'react'; import { BrowserRouter, Routes, Route } from 'react-router-dom'; const App = () => { const [darkMode, setDarkMode] = useState(false); // Check localStorage and apply theme on mount useEffect(() => { const savedTheme = localStorage.getItem('theme'); if (savedTheme === 'dark') { setDarkMode(true); document.documentElement.classList.add('dark'); } }, []); // Toggle theme and persist preference const toggleDarkMode = () => { setDarkMode(!darkMode); if (!darkMode) { document.documentElement.classList.add('dark'); localStorage.setItem('theme', 'dark'); } else { document.documentElement.classList.remove('dark'); localStorage.setItem('theme', 'light'); } }; return (<BrowserRouter> {/* Theme Toggle Button */} <button onClick={toggleDarkMode} className="fixed top-4 right-4 p-2 rounded-full bg-gray-200 dark:bg-gray-700" aria-label="Toggle Dark Mode" > {darkMode ? '🌙' : '☀️'} </button> <Routes> {/* Route definitions */} </Routes> </BrowserRouter>); }; export default App; ``` This approach

ensures: · Persistent theme preference across sessions · Immediate visual feedback on toggle · Clean separation of UI state and DOM manipulation · Compatibility with global CSS-based dark mode styling ### [**7.3.3 Database Schema Examples**]() [Figure]()5: Database collections within the MongoDB Atlas cluster. [Figure]()6: Database collections within the MongoDB Atlas cluster. ### [Explanation]() The SFT platform utilizes a document-oriented database architecture implemented using **MongoDB** and managed through MongoDB Atlas. The production database consists of eight primary collections: · `<span>users</span>` · `<span>transactions</span>` · `<span>budgets</span>` · `<span>goals</span>` · `<span>bills</span>` · `<span>wallets</span>` · `<span>p2pTransfers</span>` · `<span>notifications</span>` Each collection encapsulates logically grouped financial data aligned with domain-driven design principles. The `<span>transactions</span>` collection currently stores over 150 documents representing categorized financial activities (income and expenses) associated with individual users. The `<span>budgets</span>` collection maintains category-based spending limits and threshold alerts, enabling real-time budget monitoring. The document-based schema facilitates flexible schema evolution while maintaining referential consistency through user-specific identifiers. ### [**7.3.4 User Interface Screenshots**]() The following screenshots illustrate core user interface components of the SFT platform. **Login page** [Figure]()7Authentication interface supporting credential-based login and persistent session handling. **Analytics Page** [Figure]()8: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts (1). [Figure]()9: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts. (2) [Figure]()10: Financial overview dashboard displaying aggregated income, expenses, balance metrics, and interactive budget analytics visualized using recharts. (3) **Transactions Page** [Figure]()11: Transaction management interface with dynamic filtering (date range, category) and paginated result rendering. **Budget Page** ![]

(file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image028.gif) [Figure]()12 :Category-specific budget tracking with real-time percentage utilization indicators. [Figure]()13: generate budget from income ![] (file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image032.gif) [Figure]()14: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button **Transfers Page** ![] (file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image034.gif) [Figure]()15: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button ![] (file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image036.gif) [Figure]()16: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button ![] (file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image038.gif) [Figure]()17: P2P Transfer modal with recipient selection, amount input, and "Transfer Now" button **Dark Mode ** ![] (file:///C:\Users\ASUS\AppData\Local\Temp\msohtmlclip1\01\clip_image040.gif) [Figure]()18: Dark theme implementation demonstrating visual consistency across UI components. ## [**7.4 Challenges Encountered and Solutions**]() The development lifecycle revealed several architectural, infrastructural, and transactional challenges. Each issue required systematic root cause analysis and implementation of robust mitigation strategies. | Challenge | Problem | Root Cause | Solution | Implementation Snippet / Impact | | --- | --- | --- | --- | --- | | Challenge 1: Guest User Authentication Failure | Unauthenticated users attempting to access the demo dashboard received HTTP 401 Unauthorized responses due to JWT validation enforced at middleware level. | The authentication middleware treated all requests lacking valid JSON Web Tokens (JWTs) as unauthorized, without distinguishing guest-access scenarios. | An in-memory guest token mechanism was introduced. Temporary JWTs are generated with a restricted role (Guest) and a one-hour expiration period. Route-level authorization ensures read-only access. | Implementation Snippet:const guestToken = jwt.sign({ id: 'guest-user', role: 'Guest' }, process.env.JWT_SECRET, {

expiresIn: '1h' });This approach preserves centralized authentication enforcement while enabling controlled demonstration access. | | Challenge 2: Dark Mode Flash on Initial Render | A visual “flash of unstyled content” (FOUC) occurred when users refreshed the page with dark mode enabled. | The application (built with React) rendered the default light theme before retrieving stored theme preferences from localStorage. | A synchronous inline script was injected into index.html, executed before React hydration. This script applies the dark class to the root<html>element prior to component mounting. | Implementation Snippet:<script> if (localStorage.getItem('theme') === 'dark') { document.documentElement.classList.add('dark'); }</script>This eliminated perceptible theme flickering and improved perceived performance. | | Challenge 3: P2P Transfer Atomicity Risk | Initial transfer logic performed debit and credit operations separately. Partial execution risked financial inconsistency if server failure occurred mid-process. | Database operations were not encapsulated within ACID-compliant transactions. | Multi-document transactions were implemented using Mongoose sessions over MongoDB’s native ACID support. All wallet updates execute within a single transactional session, ensuring atomic commit or rollback. | Impact:• Eliminated risk of double-spending• Guaranteed financial data integrity• Aligned with transactional consistency requirements in financial systems | | Challenge 4: Scheduled CRON Job Failures | Bill reminders scheduled using node-cron executed inconsistently. | Timezone misconfiguration and scheduler reinitialization failure after server restarts. | 1. Standardized scheduling in UTC timezone2. Added/api/health/cronendpoint for monitoring3. Implemented scheduler reinitialization during application startup | Implementation Snippet:const cron = require('node-cron');const scheduleBillReminders = cron.schedule('0 9 * * *', async () => { await sendUpcomingBillReminders(); }, { timezone: 'UTC' });scheduleBillReminders.start(); | | Challenge 5: Cold Start Latency on Free Hosting Tier | Backend API deployment on Render free tier resulted in service suspension after inactivity, causing 8–15 second response delays. | Automatic spin-down policy applied to inactive services. | • External keep-alive pings via cron-job.org• Frontend loading state indicating server wake-up• Documented upgrade path to paid hosting tier | This mitigated user-facing latency while maintaining zero-cost deployment during development. | ## ## ## ## ## ## ## ## ## ## [**7.5 Current System Limitations**]() Despite significant functional maturity, the SFT platform retains several documented constraints. | **No.** | **Limitation** | **Description** | **Future Direction** | | ---------- | ---------- | ---------- | ---------- | | 1 | **Manual Transaction Entry** | Unlike commercial applications such as Mint and You Need A Budget (YNAB), the system

does not support automatic bank synchronization. | Optional integration via Plaid API. | | 2 | **Single-Currency Storage Model** | Multi-currency tracking is partially implemented. | Real-time conversion via Fixer.io. | | 3 | **Limited Mobile-Native Features** | The web application is responsive but lacks PWA offline capabilities and push notifications. | — | | 4 | **Forecasting Data Requirements** | Linear regression models require ≥30 days of transaction data, limiting value for new users. | — | | 5 | **Email-Only Notifications** | Notifications are delivered exclusively via SendGrid. | SMS alerts via Twilio. | | 6 | **No Investment Portfolio Tracking** | The system scope excludes equities, ETFs, or cryptocurrency monitoring. | — | | 7 | **Guest User Functional Restrictions** | Guest accounts are read-only, limiting experiential evaluation. | — | | 8 | **Test Coverage Gaps** | Current automated test coverage: 78%. Critical financial modules require expanded integration testing. | Increase coverage through additional integration tests. | | 9 | **Performance Degradation on Large Datasets** | Aggregation queries degrade beyond 1,000 transactions without indexing optimization. | Implement database indexing and query optimization. | | 10 | **Lack of Data Export** | CSV/PDF export functionality is not implemented. | Implement CSV and PDF export capabilities. | . Table 7.X: Current System Limitations and Future Directions ## [Concluding Note]() Documenting these challenges and limitations ensures methodological transparency, strengthens academic rigor, and provides a structured roadmap for future system evolution. # [Chapter 08: Discussion]() ## [Summary of the Report]() This interim report presented the architectural design, methodological framework, algorithmic foundations, and implementation progress of the Smart Financial Manager (SFT) platform. The system integrates behavioural finance principles with full-stack MERN architecture to deliver an interactive financial management solution tailored to student users. Core system components include structured transaction management, predictive expense forecasting, anomaly detection, a multi-factor Financial Health Score, peer-to-peer wallet functionality, and a banking-standard EMI calculator. The adoption of Agile Scrum enabled iterative refinement across ten structured sprints, with over 80% feature completion achieved by Sprint 8. ## [What Has Changed from the Proposal]() Several refinements were introduced during development: multi-currency support (34% user need), weighted Financial Health Score algorithm, Redis caching for sub-300ms response times, and deeper MongoDB ACID implementation for P2P transfers. These demonstrate adaptive Agile governance. ## [Future Plans and Upcoming Work]() The remaining development phases will prioritise: · Completion of integration testing and performance benchmarking · Refinement of anomaly detection thresholds · Expanded unit test coverage beyond 80% · Final usability testing against WCAG 2.1 AA standards · Documentation finalisation and deployment validation The final submission will include

empirical performance metrics, full usability evaluation results, and validation of algorithmic accuracy against test datasets. # [*References*]() Federal Reserve , 2024. *2024 Findings: Diary of Consumer Payment. * [Online]. Alea IT Solutions, 2025. *67+ Best Mobile App Development Ideas. * [Online] Available at: https://www.aleaitsolutions.com/ [Accessed 24 February 2026]. Bassett, B., 2024. *Behavioral Design in Finance. * [Online] Available at: https://www.nngroup.com/articles/behavioral-design/ [Accessed 25 February 2026]. Bitcot, 2026. *Best Tech Stack for Web and App Dev. * [Online] Available at: https://www.bitcot.com/best-tech-stack-for-web-and-app-development/ [Accessed 1 March 2026]. BrowserStack, n.d. *NFRs: What is Non Functional Requirements. * [Online] Available at: https://www.browserstack.com/guide/what-are-non-functional-requirements [Accessed 1 March 2026]. Byrne, A. & Brooks, M., 2008. *Behavioral finance: Theories and evidence. * [Online] Available at: https://www.cfainstitute.org/research/foundation/2008/behavioral-finance [Accessed 1 March 2026]. CSAC, 2025. *2024-25 SEARS Insights Brief. * [Online] Available at: https://www.csac.ca.gov/research [Accessed 2 March 2026]. Deloitte, 2023. *Digital financial ecosystems and PFM. * [Online] Available at: https://www2.deloitte.com/us/en/insights/industry/financial-services/financial-services-industry-outlooks.html [Accessed 26 February 2026]. Dey, S., 2026. *7 Mobile UX/UI Design Patterns. * [Online] Available at: https://www.interaction-design.org/literature/article/mobile-ux-design-key-principles [Accessed 27 February 2026]. Elementor, n.d. *How Accessibility Is Measured. * [Online] Available at: https://elementor.com/blog/website-accessibility/ [Accessed 27 February 2026]. Enfin Technologies, n.d. *Why MERN Stack is the Best Choice for Your Next Project?. * [Online] Available at: https://www.enfintechnologies.com/why-mern-stack-is-the-best-choice/ [Accessed 2 March 2026]. Experian, 2026. *Best Budgeting Apps of 2026. * [Online] Available at: https://www.experian.com/blogs/ask-experian/best-budgeting-apps/ [Accessed 2 March 2026]. Fixer.io, n.d. *Currency Data API Documentation. * [Online] Available at: https://fixer.io/documentation [Accessed 2 March 2026]. Kahneman, D., 2011. *Thinking, Fast and Slow. * [Online]. MongoDB Inc., 2024. *MongoDB Atlas Documentation. * [Online] Available at: https://www.mongodb.com/docs/atlas/ [Accessed 22 February 2026]. MyBankTracker, 2024. *Best Mint alternatives: Top budgeting apps compared. * [Online] Available at: https://www.mybanktracker.com/blog/budgeting/best-mint-alternatives [Accessed 2 March 2026]. NCAN, 2024. *Financial Realities Undermine Success. * [Online] Available at: https://www.ncan.org/news/financial-hardship-report-2024 [Accessed 26 February 2026]. NerdWallet, 2026. *The Best Budget Apps for 2026. * [Online] Available at: https://www.nerdwallet.com/article/finance/best-budget-apps [Accessed 2

March 2026]. OECD, 2024. *Student financial literacy: expenditure and digital financial behavior. * [Online] Available at: https://www.oecd.org/financial/education/ [Accessed 22 February 2026]. OWASP, 2023. *Role-Based Access Control (RBAC) and JWT authentication, web security standards. * [Online] Available at: https://owasp.org/ Pressman, R. & Maxim, B., 2020. *Software Engineering: A Practitioner's Approach. *9 ed. New York: McGraw-Hill Education. Redis, 2024. *How to use Redis for API Caching and Rate Limiting. * [Online] Available at: https://redis.io/docs/manual/patterns/distributed-locks/ [Accessed 1 March 2026]. Santander UK, 2025. *Santander UK finds that millions of young people still leave school without financial education. * [Online] Available at: https://www.santander.co.uk/about-santander/media-centre/press-releases/financial-education-2025 [Accessed 26 February 2026]. Schwaber, K. & Sutherland, J., 2020. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. * [Online] Available at: https://scrumguides.org/scrum-guide.html [Accessed 28 February 2026]. Sentry, 2024. *What's the difference between API Latency and API Response Time?. * [Online] Available at: https://sentry.io/answers/api-latency-vs-response-time/ [Accessed 24 February 2026]. Sommerville, I., 2021. *Engineering Software Products. * [Online]. SourceForge, n.d. *Modern MERN vs. Node.js Comparison. * [Online] Available at: https://sourceforge.net/software/compare/MERN-Stack-vs-Node-js/ [Accessed 24 February 2026]. Suryavanshi, S., 2024. *How MERN Stack Powers Modern Web Apps. * [Online] Available at: https://www.geeksforgeeks.org/mern-stack/ [Accessed 24 February 2026]. Taduka, A., 2024. *Essential NFRs for Financial Apps. * [Online] Available at: https://www.linkedin.com/pulse/non-functional-requirements-financial-applications [Accessed 26 February 2026]. Thaler, R. H. & Sunstein, C. R., 2008. *Nudge: Improving decisions about health, wealth, and happiness. *New Haven, CT: Yale University Press. World Bank, 2024. *Global Economic Prospects - Jan 2024. * [Online] Available at: https://www.worldbank.org/en/publication/global-economic-prospects [Accessed 26 February 2026]. Xiao, J. J. & O'Neill, B., 2018. *Mental accounting and behavioural hierarchy. * [Online] Available at: https://www.sciencedirect.com/science/article/abs/pii/S0167487017303215 [Accessed 2026]. --- --- --- # **APPENDIX** --- --- --- ## **Appendix A: Interview Research Materials** ### **A.1 Interview Protocol** **Research Objective** : Understand financial management practices, pain points, and expectations to inform SFT requirements. **Method** : Semi-structured interviews with 26 participants (15 students, 8 young professionals, 3 financial educators) conducted via Zoom and in-person during January-February 2026 (20-35 minutes each). Convenience sampling through university networks and LinkedIn professional groups. Thematic analysis using qualitative coding identified4 primary themes. **Ethical Considerations** : Informed

consent obtained; anonymized identifiers (S1-S15, P1-P8, E1-E3); data kept confidential; right to withdraw maintained. ### **A.2 Core Interview Questions** #### **Section 1: Current Financial Management Practices** 1. How do you currently track your income and expenses? (e.g., notebook, spreadsheet, mobile app, mental tracking, bank statements) 2. How frequently do you review your financial records? (daily, weekly, monthly, only when problems arise) 3. What categories do you use to organize your spending? (if any) 4. Have you ever used a budgeting or personal finance application? If yes, which ones and for how long? 5. What caused you to stop using those applications (if applicable)? #### **Section 2: Financial Awareness and Behavioral Patterns** 6. On a scale of 1-10, how aware are you of your daily spending? (1 = no awareness, 10 = track every transaction) 7. How often do you discover unexpected spending when checking your bank statement? 8. Do you set financial goals (e.g., saving for emergencies, specific purchases, debt reduction)? If yes, how do you track progress? 9. Have you successfully achieved a financial goal in the past year? What factors contributed to your success or failure? 10. How comfortable are you with your current level of financial control? #### **Section 3: Pain Points and Challenges** 11. What is the biggest challenge you face in managing your personal finances? 12. Do you feel mentally disconnected from your spending when using automated tracking tools or contactless payments? 13. Have you experienced "subscription fatigue" where you forget about recurring charges or subscriptions? 14. How much time per week do you spend managing your finances? Is this too much, too little, or about right? 15. Have you ever exceeded a budget without realizing it until after the fact? #### **Section 4: Feature Expectations and Preferences** 16. Would you prefer manual transaction entry or automatic bank synchronization? Why? 17. What features would make you actively use a financial management application on a daily or weekly basis? 18. How important is it to receive alerts before you overspend (proactive) versus after you overspend (reactive)? 19. Would you find value in an integrated peer-to-peer (P2P) transfer system within your financial management app for splitting bills or shared expenses? 20. Do you manage expenses in multiple currencies? If yes, how do you currently handle currency conversion tracking? 21. How valuable would predictive analytics (forecasting future expenses) be to your financial planning? #### **Section 5: Privacy, Security, and Trust** 22. What concerns, if any, do you have about connecting your bank accounts to third-party financial applications? 23. On a scale of 1-10, how important is data privacy and security when choosing a financial management tool? 24. Would you be more comfortable with a system that requires manual data entry (no bank access) versus one that automatically syncs with your bank? 25. What security features would make you trust a financial management platform? #### **Section 6: Open-Ended Insights** 26. If you could design your ideal

financial management tool, what would it look like and what capabilities would it have? 27. What would motivate you to consistently use a financial tracking system for more than three months? 28. Is there anything else about your financial management practices or needs that we haven't discussed? ### **A.3 Sample Participant Excerpt** **Excerpt - Active Awareness Gap (Participant S2, Postgraduate Student)** : *Interviewer* : "You mentioned Google Sheets. How often do you update it?" *S2* : "Honestly, maybe once a month? By the time I sit down to enter everything, I've forgotten half of what I spent. I'll look at my bank statement and think, 'Wait, did I really spend £80 on food last week?' It's always a shock. If I had to enter it right after each purchase, I think I'd be way more careful. It's like writing notes versus typing—when you physically write something, you remember it better. Same applies to finances." This quote exemplifies the recurring theme of diminished financial awareness due to delayed or automated tracking, validating the SFT platform's emphasis on immediate manual entry nudges. ### **A.4 Key Findings** **Primary Themes** (detailed in Chapter 2.1): 1. **Active Awareness Gap** : Automation reduces cognitive engagement, fostering passive spending 2. **Subscription Fatigue** : 78% forget recurring payments, causing financial leakage 3. **Privacy Concerns** : 73% reluctant toward bank integration; prefer manual entry for control 4. **Fragmented Ecosystems** : 87% use separate P2P apps, creating incomplete financial visibility **Saturation** : Thematic saturation achieved after 21 interviews; final 5 yielded no new themes, validating sample sufficiency. **End of Appendix A** --- ## **Appendix B: Comparative Analysis of Financial Tracking Systems** ### **B.1 Overview** This appendix provides a structured comparison of existing financial tracking approaches, identifies critical gaps in current solutions, and demonstrates how the Smart Financial Tracker (SFT) platform addresses these limitations through targeted design and implementation decisions. ### **B.2 Comparative Analysis Table** The following table systematically evaluates six key areas of personal financial management, comparing industry-standard approaches with identified deficiencies and the SFT platform's innovative solutions. | **Area of Analysis** | **Existing Approaches** | **Identified Gap** | **SFT Platform Contribution** | | --- | --- | --- | --- | | **Financial Tracking Approach** | Many existing systems prioritise automated transaction syncing and integration with external financial services to reduce manual input. | Automation-focused designs can reduce user engagement and limit behavioural awareness, particularly for non-technical users. | The system emphasises manual and semi-manual transaction entry to encourage active user involvement and financial awareness. | | **Behavioural Financial Awareness** | Financial applications commonly focus on recording and summarising historical transaction data. | Limited support is provided for behavioural reflection or habit improvement based on spending patterns. | Interactive dashboards and

contextual summaries are used to support behavioural understanding and informed decision-making. | | **Budgeting Support** | Most tools support monthly or long-term budgeting structures. | Short-term budgeting needs, such as weekly spending control, are insufficiently addressed. | The system incorporates both weekly and monthly budgeting with continuous monitoring. | | **Data Visualisation** | Financial data is often presented using dense or visually complex dashboards. | Excessive visual complexity can overwhelm users and reduce clarity. | Minimal, purpose-driven visualisations are adopted to enhance clarity and insight. | | **System Responsiveness** | Many systems rely on fragmented architectures and periodic data updates. | Delayed feedback limits real-time financial awareness and timely decision-making. | A unified system architecture enables near real-time updates and consistent data flow. | | **Development Methodology** | Traditional, rigid development approaches are commonly applied. | Limited user alignment and late usability feedback reduce system adoption. | An iterative, Agile-inspired development approach supports continuous refinement and user alignment. | ### **B.3 Key Insights** 1. **Manual Entry as Cognitive Engagement**: While industry trends favor automation, research demonstrates that active transaction recording strengthens financial awareness and accountability. 2. **Real-Time Feedback Mechanisms**: The SFT platform's unified architecture eliminates synchronization delays, providing immediate budget utilization updates and spending alerts. 3. **User-Centered Design**: By prioritizing clarity over feature complexity, the system reduces cognitive load and supports sustained user engagement. 4. **Behavioral Reinforcement**: Progressive threshold alerts (80%, 90%, 100%) enable proactive spending correction rather than reactive damage control. 5. **Agile Development**: Continuous user feedback integration ensures alignment with real-world financial management needs rather than theoretical feature sets. **End of Appendix B** --- ## **Appendix C: Quantitative Research Questionnaire – Financial Behaviors of Emerging Adults** ### **C.1 Research Overview** This appendix contains the structured questionnaire administered to assess financial behaviors, literacy levels, hardship indicators, and spending patterns among emerging adults (ages 18-24), with particular focus on university students and early-career professionals. The survey was conducted between December 2025 and January 2026, with responses collected from 156 participants across multiple institutions. ### **C.2 Research Methodology** **Survey Design**: Closed-ended quantitative questionnaire with Likert scales and categorical responses **Administration Method**: Online survey platform (Google Forms / Qualtrics) with anonymous response collection **Sample Size**: 156 respondents (98 university students, 42 early-career professionals, 16 recent graduates) **Duration**: 8-12 minutes average completion time **Data Analysis**: Descriptive statistics, frequency distributions, cross-tabulation analysis ###

C.3 Survey Questionnaire #### **Section 1: Demographic Information** **Q1.1** What is your age group? - 18-20 years - 21-24 years - 25-27 years - 28+ years **Q1.2** What is your current status? - Full-time university student - Part-time university student - Recent graduate (within 1 year) - Early-career professional (1-3 years post-graduation) - Other: _____ **Q1.3** What is your primary source of income? - Family support/allowance - Part-time employment - Full-time employment - Student loans/grants - Freelance/gig work - Multiple sources - No regular income **Q1.4** What is your approximate monthly income (including all sources)? - Less than $500 - $500 - $1,000 - $1,001 - $1,500 - $1,501 - $2,000 - $2,001 - $3,000 - More than $3,000 - Prefer not to say --- #### **Section 2: Financial Hardship Assessment** **Q2.1** If you faced an unexpected $500 expense today (e.g., medical emergency, car repair), could you cover it without borrowing money? - Yes, easily - Yes, with some difficulty - No, I would need to borrow - No, I could not cover it at all **Q2.2** In the past 12 months, have you completely run out of money before the end of the month? - Never - Once or twice - 3-5 times - 6-8 times - More than 8 times **Q2.3** In the past 6 months, have you experienced any of the following due to financial constraints? (Select all that apply) - Skipped meals to save money - Could not afford required textbooks or course materials - Delayed or avoided medical/dental care - Had difficulty paying rent or utilities - Could not afford basic toiletries or hygiene products - None of the above **Q2.4** How often do you worry about your financial situation? - Never - Rarely (once a month or less) - Sometimes (2-3 times per month) - Often (weekly) - Daily **Q2.5** Has financial stress affected your academic performance or work productivity in the past 6 months? - Not at all - Slightly - Moderately - Significantly - I had to suspend studies/work due to financial issues --- #### **Section 3: Financial Literacy and Confidence** **Q3.1** How confident do you feel about managing your personal finances? - Very confident - Somewhat confident - Neutral - Not very confident - Not confident at all **Q3.2** Have you ever created a formal budget or spending plan? - Yes, and I follow it regularly - Yes, but I don't follow it consistently - I tried once but stopped - No, I have never created a budget **Q3.3** Do you currently track your income and expenses? - Yes, using a budgeting app - Yes, using spreadsheets - Yes, using pen and paper - Sometimes, informally - No, I don't track my finances **Q3.4** Do you have an emergency fund (savings set aside for unexpected expenses)? - Yes, covering 3+ months of expenses - Yes, covering 1-2 months of expenses - Yes, but less than 1 month of expenses - No, but I am working on building one - No, I don't have any emergency savings **Q3.5** Where do you primarily learn about personal finance management? (Select up to 2) - University courses or workshops - Parents or family members - Social media (TikTok, Instagram, YouTube) - Financial blogs or websites - Books or podcasts - Financial advisors or professionals - Friends or peers - I don't

actively seek financial education --- #### **Section 4: Spending Behavior and Patterns** **Q4.1** On average, what percentage of your monthly income do you save? - 0% (I spend everything or more) - 1-10% - 11-20% - 21-30% - More than 30% **Q4.2** Do you have recurring digital subscriptions (streaming services, apps, gym memberships, etc.)? - No subscriptions - 1-2 subscriptions - 3-5 subscriptions - 6-10 subscriptions - More than 10 subscriptions **Q4.3** Can you accurately estimate your total monthly subscription costs? - Yes, I know the exact amount - Yes, within $10-20 - Somewhat, within $50 - No, I would underestimate significantly - I don't pay attention to subscription costs **Q4.4** How often do you make impulse purchases (unplanned buying)? - Never - Rarely (once a month or less) - Sometimes (2-3 times per month) - Frequently (weekly) - Very frequently (multiple times per week) **Q4.5** Which spending category consumes the largest portion of your budget? (Select one) - Food (groceries and dining out) - Housing (rent, utilities) - Transportation - Entertainment and social activities - Shopping (clothing, electronics) - Education (tuition, books, supplies) - Debt payments - Other: _____ --- #### **Section 5: Technology and Financial Tools** **Q5.1** Do you currently use any personal finance management applications? - Yes, regularly - Yes, but inconsistently - I tried but stopped using them - No, but I am interested - No, and I am not interested **Q5.2** If you don't use budgeting apps, what prevents you? (Select all that apply) - Too complicated to use - Privacy concerns about bank integration - Cost/subscription fees - Don't see the value - Prefer manual tracking methods - Don't have time to set it up - Other: _____ - Not applicable (I use budgeting apps) **Q5.3** Which features would be most valuable in a personal finance app? (Rank top 3) - Manual transaction entry - Automated bank synchronization - Budget tracking with alerts - Spending visualizations and charts - Goal tracking for savings - Bill reminders - Peer-to-peer money transfers - Financial health score - Multi-currency support **Q5.4** How often would you realistically use a financial tracking tool? - Daily - Several times per week - Weekly - Monthly - Rarely/as needed - I would not use it --- #### **Section 6: Future Financial Goals and Concerns** **Q6.1** What is your primary financial goal for the next 12 months? (Select one) - Build an emergency fund - Reduce/eliminate debt - Save for a specific purchase (car, travel, etc.) - Improve spending discipline - Increase income - Learn personal finance skills - No specific goal **Q6.2** Which financial challenge concerns you the most? - Student loan debt - Not earning enough income - Unable to save money - Poor spending habits - Lack of financial knowledge - Economic uncertainty/inflation - Future career prospects **Q6.3** Would you be interested in using a free, privacy-focused financial tracking platform designed specifically for students and young professionals? - Definitely yes - Probably yes - Maybe/Unsure - Probably not - Definitely not ### **C.4 Data Analysis Summary** **Response Rate**: 156 completed surveys out of 180 distributed (86.7% completion

rate) **Key Statistical Findings**: - 56% could not cover a $500 emergency expense - 68% ran out of money at least once in the past year - 67% reported skipping meals due to financial constraints - 80% expressed confidence in money management despite 80% never creating a formal budget - 77% lack emergency fund savings - 25% rely on social media as primary source of financial education - 34% underestimate recurring subscription costs by more than 30% **Research Validation**: Survey results align with contemporary literature on financial stress among emerging adults (NCAN, 2024; OECD, 2024; Santander UK, 2025) and directly informed the functional requirements and behavioral design principles of the SFT platform. **End of Appendix C**